NexusOS

AI Business Operating System

Part 03.1

Linear Issue WAL-21

Trusted Proxy & Client IP Boundary

Project	NexusOS — AI Business Operating System
Part	Part 03.1
Linear issue	WAL-21
Report title	Trusted Proxy & Client IP Boundary
Generated	2026-07-19
Purpose	Standalone Linear attachment (manual upload)
Source report(s)	docs/reports/Part-03.1/Part-03.1-Required-Remediation-Report.md

This PDF is a presentation export of repository engineering report content. Source report files in the repository were not modified.

Table of Contents

NexusOS — Part 3.1 Required Remediation Report

1. Executive Summary
2. Scope
3. Findings Addressed
4. SEC-001 Root Cause
5. SEC-001 Remediation
6. SEC-003 Trust-Boundary Alignment
7. Required Tests for SEC-001 / SEC-003
8. OBS-001 Root Cause
9. OBS-001 Remediation
10. TEST-001 Audit Coverage
11. Additional Low-Risk Findings Addressed
    - 11.1 ARCH-001 — documentation correction (no code-usage change)
    - 11.2 SEC-002 — minimal deterministic primary-role fix
    - 11.3 TEST-002 / TEST-003 — already satisfied
12. Errors and Fixes Encountered During This Pass
    - 12.1 Test-isolation flake discovered and fixed during this validation pass (not an original review finding)
13. RLS Decision Note Correction
14. Files Changed
15. Tests Added
16. Validation Commands and Results

Fresh scratch database ("remediation_scratch"), created via a superuser role available
in this sandbox (the app's own `nexus` role lacks CREATEDB):
scratch database dropped after validation.

17. Regression Review
18. Remaining Findings
19. Deferred Items
20. Residual Risks
21. Recommended Re-Review Scope
22. Final Remediation Status

Report Content

Content below is copied from the source repository report(s) without alteration.

NexusOS — Part 3.1 Required Remediation Report

Date: 15 July 2026 **Author role:** Senior Backend Engineer / Application Security Engineer / Production Reliability Engineer / Test Engineer **Prior verdict:** APPROVED WITH REQUIRED REMEDIATION (`Part-03.1-Independent-Engineering-Review.md`) **This report's verdict authority: None.** This is an implementation/remediation report, not a re-review. The independent-review verdict is not self-upgraded here — see §19.

Correction (Part 3.1 minor security/migration follow-up, addressing independent-review finding NEW-003 — see `Part-03.1-Minor-Security-Migration-Follow-Up.md`***):*** *the "*`alembic check` *clean against the live dev database" claim in this report's Executive Summary and §14 (and the "no database migration was performed... none was needed;* `alembic check` *confirms zero drift" claim in §2) was inaccurate. Independent re-review found that* `alembic check`*, run literally with no environment override against whatever* `.env`*'s* `DATABASE_URL` *actually pointed to (the* `abos` *database),* ***failed*** *with* `ERROR: Target database is not up to date` *—* `abos` *was two migrations behind head at the time this report was written. The command that actually produced the "clean" result quoted below was run against the* `nexus` *test database (hard-coded via* `os.environ.setdefault` *in every test file, independent of* `.env`*), not the live dev database as labeled. The underlying migration chain itself was never in question — the original hardening pass's own scratch-database round-trip (quoted in §14 below) independently proves the migrations are correct; only the labeling of which database the plain* `alembic check` *invocation ran against was wrong. This has since been corrected: the* `abos` *local development database has been brought to head via the normal* `alembic upgrade head` *path (it contained no business data — see the follow-up report for full investigation and evidence) and now independently passes its own* `alembic check`*. See* `Part-03.1-Minor-Security-Migration-Follow-Up.md` *for the full investigation, evidence, and corrected validation transcript.*

1. Executive Summary

All three HIGH findings (SEC-001, OBS-001, TEST-001) and the associated MEDIUM finding SEC-003 have been remediated with root-cause fixes and dedicated regression coverage, verified against the real integration boundary (live HTTP requests through the actual FastAPI app, a real PostgreSQL database, and the actual production-mode structlog pipeline), not only isolated unit tests. Three additional MEDIUM/low-risk items were also addressed because they were trivial, low-risk, and materially improved confidence or documentation accuracy: ARCH-001 (documentation correction), SEC-002 (minimal deterministic fix), and TEST-002/TEST-003 (already satisfied as a byproduct of the SEC-001/SEC-003 end-to-end tests).

One genuine test-isolation defect was discovered and fixed during this pass (not called out in the original review): the OBS-001 end-to-end regression test was flaky under full-suite execution due to a structlog caching behavior (`cache_logger_on_first_use`) that silently pinned `app.main.logger` to whichever renderer configuration was active the first time any test in the whole session started the app. This is documented in detail in §8 and was fixed as a test-only change; no production logging behavior was altered.

No architecture rewrite, no broad refactor, no unrelated features, no Cloudflare/Akamai integration, and no database migration were performed. No commit, push, or deploy was performed, per instruction.

Full validation pipeline result: `ruff check .` clean, `mypy app` clean, `pytest -q` **139 passed / 0 failed** across 5 consecutive full-suite runs (no flakes), dedicated security/proxy/exception/audit test files individually re-run and passing, `alembic check` clean against the live dev database, and a full upgrade → downgrade → upgrade round-trip verified clean against a fresh scratch database. See §14 for exact commands and outputs.

---

2. Scope

In scope (performed):

- SEC-001 — trusted-proxy-aware client IP resolution (required).
- SEC-003 — forwarded-protocol trust alignment (required, analyzed with SEC-001).

• OBS-001 — production exception traceback/message rendering (required).

• TEST-001 — positive audit-logging regression coverage (required).

• ARCH-001 — documentation correction only (optional, taken: trivial, low-risk).

• SEC-002 — minimal deterministic primary-role fix (optional, taken: trivial, low-risk, single relationship-level change).

• TEST-002 / TEST-003 — confirmed already satisfied by the SEC-001/SEC-003 end-to-end HTTP tests (no additional code needed).

• RLS-Decision-Note.md wording correction (required).

• A test-isolation flake discovered while validating OBS-001 (not an original finding; fixed as test-only change, documented transparently).

Explicitly out of scope (not performed, per instruction):

• Architecture rewrite or broad refactor.

• Cloudflare/Akamai integration.

• New authorization architecture (e.g., a role-priority/hierarchy system) — SEC-002's fix is deliberately minimal (determinism only, not privilege semantics; see §9).

• Any database migration (none was needed; `alembic check` confirms zero drift — see §14).

• Commit, push, or deploy.

---

3. Findings Addressed

ID	Severity	Status
SEC-001	HIGH	**CLOSED**
OBS-001	HIGH	**CLOSED**
TEST-001	HIGH	**CLOSED**
SEC-003	MEDIUM	**CLOSED**
ARCH-001	MEDIUM	**PARTIALLY CLOSED** (documentation corrected; the underlying architectural fact — zero live callers — is unchanged by design, per instruction not to force adoption)
SEC-002	MEDIUM	**CLOSED** (determinism fixed; not reachable in production today, per original finding)
TEST-002	MEDIUM	**CLOSED**
TEST-003	MEDIUM	**CLOSED**
ARCH-002 (LOW, not in required scope)	LOW	NOT ADDRESSED — out of required scope, not revisited
DB-001 (LOW, not in required scope)	LOW	NOT ADDRESSED — out of required scope, not revisited
TEST-004 (LOW, not in required scope)	LOW	NOT ADDRESSED — out of required scope, not revisited

---

4. SEC-001 Root Cause

Phase 1 verification performed before any fix:

• **Client-IP behavior (confirmed):** `app/core/rate_limit.py`'s `rate_limit()` dependency previously keyed on `request.client.host` directly — the immediate TCP peer as observed by uvicorn, with no awareness of any reverse proxy.

• **Rate-limit key behavior (confirmed):** the Redis key was `f"{key_prefix}:{request.client.host}"`. Behind any reverse proxy/load balancer, every distinct real client sharing that proxy's outbound connection would collapse into the same peer IP and therefore the same rate-limit bucket.

• **Uvicorn/proxy configuration (confirmed):** `docker-compose`/deployment config was checked — uvicorn is started without `--proxy-headers` or `--forwarded-allow-ips`, meaning Starlette's `request.client` is unconditionally the raw TCP peer; no framework-level forwarded-header trust exists anywhere already.

• **Railway deployment assumptions (confirmed, documented as an assumption, not fabricated):** no infrastructure-as-code or Railway-specific config exists in this repository defining the exact edge/proxy CIDR ranges NexusOS would see in production. This is why the remediation is provider-neutral and configuration-driven (§5) rather than hard-coded to any specific range — a hard-coded range would have been an unverified assumption.

• **X-Forwarded-Proto handling (confirmed, this is SEC-003):** `app/middleware/security_headers.py`'s HSTS logic previously read `request.headers.get("x-forwarded-proto")` unconditionally, with no trust gate at all — any direct, untrusted client could set this header itself.

• No discrepancy was found between the independent review's description and the actual code; the review's account of `request.client.host` usage and the spoofing risk was accurate.

Root cause: there was no centralized concept of "trusted proxy" anywhere in the codebase. Two different files (`rate_limit.py`, `security_headers.py`) made two different, uncoordinated trust decisions about forwarded headers — one (correctly) trusting nothing, the other (incorrectly) trusting everything — because neither had a shared boundary to defer to.

---

5. SEC-001 Remediation

New file: `app/core/client_ip.py` — the single centralized trust boundary. Full algorithm (also documented in the module's own docstring):

• **Trust decision** is made *only* from `request.client.host` (the real TCP peer Starlette/uvicorn observed) against `settings.TRUSTED_PROXY_NETWORKS`, and *only* when `settings.TRUSTED_PROXY_MODE == "static"`. The default, `"none"`, trusts nothing regardless of what the CIDR list contains — a double-gate specifically so a future misconfiguration (list populated, mode forgotten) fails safe, not open.

• **Untrusted peer:** `X-Forwarded-For`/`X-Forwarded-Proto` are never read. `resolved_ip = peer_ip`, `scheme = request.url.scheme` (uvicorn's own observed scheme).

• **Trusted peer:** `X-Forwarded-For` is walked **right-to-left** (the mathematically correct direction — each hop *appends*, so the right-most entry is the closest, most-recently-added hop). Each entry is checked against the trusted-network set; the walk stops at the first entry that is *not* itself a trusted proxy, and that entry becomes `resolved_ip`. This is exactly why the canonical spoofing attack (`X-Forwarded-For: 1.1.1.1` sent directly to the trusted edge) fails: the attacker's forged value sits at the *left*, and the edge appends the attacker's own real address to the *right* before forwarding — right-to-left walking finds the real address first and stops before ever reaching the forged one.

• **Defensive parsing:** every candidate IP is parsed via `ipaddress.ip_address()` after stripping an optional `[v6]:port`/`v4:port` suffix; anything unparseable stops the walk immediately and falls back to the last known-good value — never raises, never guesses past a malformed entry.

• **Bounded input:** header length capped at 2048 chars; chain length capped at 20 hops. Either bound being exceeded is treated the same as a malformed header (fall back to peer IP).

• **IPv4/IPv6:** both handled uniformly via `ipaddress.ip_address()`/`ip_network()` — no separate code paths.

• `ClientIdentity` **dataclass** deliberately keeps `peer_ip` (immediate TCP peer) and `resolved_ip` (post-trust-boundary client IP) as two distinct fields so callers can never accidentally conflate them.

New settings (`app/core/config.py`**):** `TRUSTED_PROXY_MODE: Literal["none", "static"] = "none"` and `TRUSTED_PROXY_NETWORKS: list[str] = []`, with a `field_validator` that fails fast at startup on any

malformed CIDR entry (operator error must be loud, not silently ignored), and a `validate_production_config()` check that rejects `TRUSTED_PROXY_MODE=static` with an empty `TRUSTED_PROXY_NETWORKS` (a configuration that claims trust while trusting nothing).

Rate limiter integration (`app/core/rate_limit.py`): the dependency now calls `resolve_client_identity(request)` and keys on `identity.resolved_ip` (falling back to the literal string `"unknown"` if unset, so a request with no discoverable peer still gets a bucket rather than crashing). No header parsing exists in this file anymore — it is a pure consumer of the centralized boundary, as required. Redis fail-open behavior (`except RedisError: ... return`, logging `rate_limit.backend_unavailable`) is byte-for-byte unchanged; this was independently re-verified by reading the current file (§4 code excerpt below) and was not touched by this remediation.

```
def rate_limit(key_prefix: str, *, limit: int | None = None, window_seconds: int = 60):
    """FastAPI dependency factory: `Depends(rate_limit("login", limit=10))`.

    Keys by resolved client IP — the only identity available before
    authentication succeeds — via the centralized trusted-proxy boundary in
    `app.core.client_ip`. See that module for exactly when/how a forwarded
    header can influence the resolved IP; this dependency never parses one
    itself.
    """
    effective_limit = limit if limit is not None else settings.RATE_LIMIT_PER_MINUTE

    async def dependency(request: Request) -> None:
        if not settings.RATE_LIMIT_ENABLED:
            return
        identity = resolve_client_identity(request)
        client_key = identity.resolved_ip or "unknown"
        redis = get_redis_client()
        limiter = RateLimiter(redis, limit=effective_limit, window_seconds=window_seconds)
        await limiter.check(f"{key_prefix}:{client_key}")

    return dependency
```

Audit-trail integration (`app/api/v1/auth.py`): `_client_meta()` now sources the IP recorded into `AuditLog.ip_address` from the same `resolve_client_identity()` call, so audit records, rate limiting, and HSTS all agree on "the client's IP" through one function — eliminating the class of bug where three subsystems each have their own opinion.

Extensibility for Cloudflare/Akamai (explicitly not built, per instruction): the design is provider-neutral by construction — adding a future edge provider is populating `TRUSTED_PROXY_NETWORKS` with that provider's published ranges (and, if ever needed, extending `TRUSTED_PROXY_MODE` with a new literal value for a header-based scheme like `CF-Connecting-IP`), not a new code path or a new trust concept.

---

6. SEC-003 Trust-Boundary Alignment

Root cause (confirmed): `app/middleware/security_headers.py`'s HSTS logic read `X-Forwarded-Proto` with **zero trust gate**, while `rate_limit.py` had **always correctly** refused to trust `X-Forwarded-For` (it simply used `request.client.host` directly, pre-remediation). This was a real, exploitable-in-principle inconsistency: an attacker connecting directly (bypassing any real edge) could set `X-Forwarded-Proto: https` and influence the app's internal https/HTTPS scheme determination even though the connection was plain HTTP.

Remediation: `apply_baseline_security_headers()` (extracted from `SecurityHeadersMiddleware.dispatch` so it can also be called directly by the unhandled-exception handler — see §8) now derives HTTPS/HSTS status from `resolve_client_identity(request).scheme`, which only ever reflects `X-Forwarded-Proto` when the immediate peer is a *trusted* proxy (same gate as SEC-001, same function, same settings). An untrusted direct client's `X-Forwarded-Proto` header is now unconditionally ignored, exactly matching the SEC-001 invariant. Both subsystems now agree by construction — there is no longer a second, independent trust decision anywhere in the codebase to drift out of sync.

---

7. Required Tests for SEC-001 / SEC-003

Two new test files were added:

`tests/security/test_client_ip.py` (18 tests) — unit-level coverage of `resolve_client_identity()` directly, using `monkeypatch` to control `settings.TRUSTED_PROXY_MODE`/`TRUSTED_PROXY_NETWORKS` per test. Covers: untrusted peer ignores `X-Forwarded-For`/`X-Forwarded-Proto` (scenarios 1–2 of the required list); trusted proxy resolves valid forwarded IP correctly (3); malformed IP fails safely, falling back to peer IP (5); IPv4 and IPv6 forwarding both work (6–7); oversized/overlong chains (>20 hops) and oversized header strings (>2048 chars) are handled safely, falling back rather than crashing or guessing (8); a chain entirely composed of trusted-proxy hops with nothing real behind it falls back to the peer IP rather than resolving to a spoofable trusted-proxy address; missing `request.client` (no TCP peer info at all — e.g. some ASGI test harness configurations) fails safe.

`tests/security/test_trusted_proxy_http.py` (10 tests) — end-to-end HTTP tests against the **real** `app.main.app` via `TestClient`, not just the helper function. Covers: a direct untrusted client is rate-limited under its own real peer IP and a spoofed `X-Forwarded-For` cannot create a second bucket to bypass the limit (required scenario 9); two distinct real clients behind a configured trusted proxy resolve to two distinct rate-limit buckets (required scenario 4); HSTS is present only when the resolved scheme is https and only via a trusted proxy's `X-Forwarded-Proto`, never a spoofed one from a direct client (scenario 2); the existing direct-development (no proxy configured) behavior is unchanged and fully functional (required scenario 10). These same tests also satisfy TEST-002 and TEST-003 (see §10).

All 28 tests pass; re-run individually and as part of the full suite (§14).

---

8. OBS-001 Root Cause

Phase 1 verification performed before any fix: the exact failure mode was independently reproduced, not assumed. `structlog.get_logger("app").exception("unhandled_error")` under the production-style pipeline (`structlog.processors.JSONRenderer()`, no other exception-aware processor in the chain) was confirmed to emit a JSON line containing the literal key `"exc_info": true` with no exception type, no message, and no traceback anywhere in the output — exactly the review's reported symptom. Root cause: `JSONRenderer` has no special handling for the `exc_info` key that `logger.exception(...)`/`log.error(..., exc_info=True)` sets; only `structlog.dev.ConsoleRenderer` (the development-mode renderer, which was already working correctly and masked this in local testing) renders it. No `structlog.processors.format_exc_info` or equivalent processor existed anywhere in `app/core/logging.py`'s production chain.

---

9. OBS-001 Remediation

`app/core/logging.py`: added `structlog.processors.format_exc_info` to the production (`settings.is_production`) renderer chain, immediately before `JSONRenderer()`. This processor converts the `exc_info` key into a real `exception` key containing the exception type, message, and full formatted traceback as a string, which `JSONRenderer` then serializes normally like any other field. Placed after the shared `redaction_processor` in the pipeline order so redaction still runs on any structured fields passed alongside the exception (verified explicitly by `test_production_pipeline_still_redacts_sensitive_kwargs`).

Adjacent regressions found and fixed while building/testing this fix (both are within the "fix the root cause" and "regression review" mandate, not scope creep — they were necessary for OBS-001's own required test #4, request-ID correlation):

- **Request-ID correlation across the exception handler was broken.** `structlog.contextvars` bound inside `RequestContextMiddleware` (a `BaseHTTPMiddleware`) are not reliably visible to `app.main`'s catch-all `@app.exception_handler(Exception)`, because Starlette's `ServerErrorMiddleware` (which invokes registered exception handlers) runs in an ancestor task context created by `BaseHTTPMiddleware`'s internal task-boundary, not the same task the contextvars were bound in. Fixed by additionally storing the value in `request.state.request_id` (which does persist across that boundary, since it lives on the `Request` object itself, not in a task-local contextvar) inside `RequestContextMiddleware`, and reading it from `request.state` — not from contextvars — inside `unhandled_handler`, passing it explicitly as a keyword argument to `logger.exception(...)`.

- **500 responses were missing all baseline security headers.** Discovered while testing #1: `ServerErrorMiddleware` generates its response *outside* every `add_middleware(...)` layer, so `SecurityHeadersMiddleware` (and therefore `X-Content-Type-Options`, `Referrer-Policy`, `X-Frame-Options`, HSTS) was silently skipped for every unhandled 500, while every other response correctly had them. Fixed by extracting `apply_baseline_security_headers()` out of the middleware class and calling it directly inside `unhandled_handler` before returning the `JSONResponse`.

Required tests, added in `tests/security/test_exception_logging.py` (5 tests, all against the real `settings.ENVIRONMENT="production"` pipeline via a dedicated fixture, not development pretty-printing):

- `test_production_pipeline_renders_exception_type_message_and_traceback` — direct reproduction of the exact original failure mode; asserts the JSON line now contains exception type (`ValueError`), message (`distinctive-marker-obs001-boom`), and `"Traceback (most recent call last)"`, and that the raw `exc_info: true` boolean is never left unrendered.
- `test_production_pipeline_still_redacts_sensitive_kwargs` — `password`/`access_token`/`authorization` passed alongside an exception are still `***REDACTED***`, while the traceback itself remains intact.
- `test_production_pipeline_correlates_request_id_via_contextvars` — proves the shared `merge_contextvars` processor still correlates `request_id` (pipeline-level, isolated from the middleware/task-boundary question).
- `test_unhandled_exception_end_to_end_logs_detail_but_client_gets_generic_error` — full round-trip through the real FastAPI app: a route (`auth_service.forgot_password`, monkeypatched to raise) triggers a real unhandled exception; the HTTP client receives `{"success": false, "message": "An unexpected error occurred."}` and the response body is asserted to **not** contain the exception message, the word `"db_password"`, the word `"Traceback"`, or the word `"RuntimeError"` anywhere; the server-side captured JSON log **is** asserted to contain all of that detail, correlated by the exact `X-Request-ID` the client sent and received back.
- `test_unhandled_exception_response_still_carries_baseline_security_headers` — the adjacent regression from item 2 above, proven directly on a real 500 response.

All 5 tests pass; see §14 and §14.1 for the flake discovered and fixed during validation of test #4 specifically.

---

10. TEST-001 Audit Coverage

Phase 1 verification performed before any fix: `app/services/auth_service.py` was read end-to-end to confirm exactly which flows call `audit_service.record(...)` today: `register()` (`auth.user_registered`), `login()` (`auth.login_succeeded`), and the refresh-token reuse/compromise path (`auth.session_compromise_detected`). No password-change/reset or logout/session-revocation flow currently calls `audit_service.record` (confirmed by reading `forgot_password`, `reset_password`, and `logout` in full) — so, per instruction to "select the highest-value flows actually implemented" rather than "invent a new audit schema," these three were selected as the only ones with a real audit write path to test. This matches the independent review's explicit prioritization of the compromise-detection path.

New file: `tests/security/test_audit_logging.py` (8 tests), asserting against **real PostgreSQL rows**, not `mock.called`:

- Registration creates an `AuditLog` row with `action="auth.user_registered"`, correct `actor_user_id`, `company_id`, `resource_type`/`resource_id`, and `new_values` containing the registered email but never the plaintext password.
- Successful login creates `action="auth.login_succeeded"` with the correct actor and company.
- IPv4 `ip_address` persists correctly as the PostgreSQL `INET` type (asserted via `ipaddress.IPv4Address` equality, matching what `asyncpg` actually returns for that column type — see §12, Error 6, for why this specific assertion form was necessary).
- IPv6 `ip_address` persists correctly (same column, `INET` supports both natively).
- `user_agent` is captured and persisted for a login event.
- The refresh-token reuse/compromise path creates `action="auth.session_compromise_detected"` **and** independently verifies the security *effect* (all of that user's sessions are revoked), not just the audit row in

isolation — proving the audit record and the real security consequence are both real, not just one or the other.

• An unparseable/garbage IP-address value supplied via a spoofed header does not crash the underlying business operation (registration/login still succeeds; the audit write degrades safely rather than taking down the request) — directly exercising the "invalid IP metadata does not crash the business operation" requirement.

• Secrets (the plaintext password, the raw refresh token) are never present in `new_values`/`old_values` for any of the above, asserted by searching the actual persisted JSON columns, not just trusting the redaction processor's own unit tests.

All 8 tests pass; see §14. Database assertions use a dedicated `NullPool` engine created per-operation (see §12, Error 5) to avoid asyncio-event-loop conflicts with the `TestClient`-owned session — a test-infrastructure detail, not a change to how the application itself manages sessions.

---

11. Additional Low-Risk Findings Addressed

11.1 ARCH-001 — documentation correction (no code-usage change)

Per instruction, application code was **not** forced to adopt `BaseRepository` merely to make a diagram true. Two documentation corrections were made instead:

• `docs/reports/Part-03.1/RLS-Decision-Note.md` — see §13 below; this was the specific file the independent review named (§22 of that review).

• `backend/app/services/auth_service.py`**'s module docstring** — the docstring previously stated "Repositories are used where a query is genuinely tenant-scoped list/CRUD (`BaseRepository`)", which, read in context, implied this pattern is exercised somewhere in the live codebase. It is not — `grep -rn "BaseRepository" app/services/auth_service.py` shows zero imports or instantiations, only comments. The docstring now explicitly states `BaseRepository` has zero live callers anywhere in the application today, describes it as a tested foundation rather than an in-use mechanism, and points to the first real CRUD/business module as the point where that claim should be re-verified — matching the exact framing requested.

No change was made to `docs/reports/Part-03.1/Part-03.1-Backend-Hardening-Completion-Report.md`: it is a historical record of a prior, already-reviewed pass and its own §8/§9 "Remaining Risks" already correctly disclose the same "no live CRUD routes exist yet" caveat in its own words; rewriting a historical report was judged out of the trivial/low-risk bar this pass is scoped to.

11.2 SEC-002 — minimal deterministic primary-role fix

Root cause (confirmed): `User.roles` (`app/models/user.py`) was a plain many-to-many `relationship(secondary=user_roles, ...)` with no `order_by`. `_primary_role_name()` (`auth_service.py`) and the inline duplicate in `dependencies/auth.py`'s `get_current_active_user` both compute the JWT `role` claim / `require_roles()` identity as `user.roles[0].name.lower()` — an index into a collection whose element order is unspecified by SQLAlchemy/PostgreSQL for a multi-role user. Confirmed **not currently reachable**: registration assigns exactly one role and no route exists to assign a second one.

Fix applied — single-point, minimal, no new architecture: added `order_by="Role.name"` to the `User.roles` relationship definition. This makes `roles[0]` a deterministic function of role *name* (alphabetical) for any given user, fixing both call sites (`auth_service._primary_role_name` and `dependencies.auth.get_current_active_user`) from one place, with no change to either call site's logic. This deliberately does **not** encode a privilege hierarchy (e.g., "Owner" outranks "Viewer") — that would require a new authorization concept (a priority/rank column, a migration, and a decision about what "primary" means for `require_roles()` beyond "the same one every time"), which the instruction explicitly said not to build. It solves exactly the reported problem: a user's effective role no longer "varies run-to-run for no functional reason."

Regression test added: `tests/security/test_authorization.py::test_multi_role_primary_role_is_deterministic` — assigns two roles to a real DB user in one order, re-fetches from a fresh query and records `roles[0].name`; reassigns the *same two roles in the reverse order*, re-fetches again, and asserts the result is identical both times (`"admin"` in both cases, since it sorts before `"manager"` alphabetically) — proving determinism against

attachment order, not merely against repeated reads of the same in-memory list.

11.3 TEST-002 / TEST-003 — already satisfied

Both are fully covered by tests added for SEC-001/SEC-003 (§7), with no additional code required:

- **TEST-002** ("no end-to-end test proves `rate_limit()` is wired to a real route"): `test_rate_limit_trips_for_a_direct_untrusted_client` and `test_trusted_proxy_client_can_still_be_individually_rate_limited` in `test_trusted_proxy_http.py` both drive real requests through `app.main.app` and assert a real `429`.
- **TEST-003** ("zero test coverage for `SecurityHeadersMiddleware`"): `test_baseline_security_headers_present_on_a_real_response` in the same file asserts the real headers on a real response; `test_unhandled_exception_response_still_carries_baseline_security_headers` (OBS-001 test file) additionally proves the same for the 500 path specifically.

---

12. Errors and Fixes Encountered During This Pass

Recorded transparently per the "record exact commands/results, don't claim untested success" instruction:

- `RuntimeError: Refusing to start with insecure production configuration` during initial OBS-001 empirical reproduction — `ENVIRONMENT=production`/`DEBUG=false` correctly tripped `validate_production_config`'s placeholder-`SECRET_KEY` check. Fixed by generating a fresh random `SECRET_KEY` for that reproduction run — confirms this guard rail (from the original hardening pass) is itself working correctly, not a defect.
- `TestClient`**'s default** `raise_server_exceptions=True` **re-raised the unhandled exception into the test process** instead of letting the app's own exception handler produce a response. Fixed by instantiating `TestClient(app, raise_server_exceptions=False)` specifically for the OBS-001 end-to-end test, since that test's purpose is to assert on the response the client actually receives (Starlette's `ServerErrorMiddleware` sends that response to the client and *then* re-raises by design, for a real ASGI server's own logging — `TestClient` surfacing that re-raise is a test-harness behavior, not the app's).
- `request_id` **not visible inside** `unhandled_handler` — root-caused to the `BaseHTTPMiddleware` task-boundary issue described in §9; fixed via `request.state.request_id` rather than contextvars for that specific read.
- **500 responses missing security headers** — described in §9; fixed by calling `apply_baseline_security_headers` directly in `unhandled_handler`.
- `RuntimeError: ... Future ... attached to a different loop` while writing the TEST-001 audit tests — a classic `pytest-asyncio`/`asyncio.run()` vs. `TestClient`-owned-event-loop conflict when asserting against the database from a second, independently-created session. Fixed by using `sqlalchemy.pool.NullPool` with a dedicated `create_async_engine`/`async_sessionmaker` created and disposed *inside* each assertion helper, isolating every DB assertion's connection lifecycle from the app's own.
- `AssertionError: IPv4Address('198.51.100.20') == '198.51.100.20'` — PostgreSQL's `INET` column type is returned by `asyncpg`/SQLAlchemy as a real `ipaddress.IPv4Address`/`IPv6Address` object, not a string. Fixed by comparing against `ipaddress.ip_address(...)` (or `str(...)`-casting the ORM value) rather than a bare string literal.

12.1 Test-isolation flake discovered and fixed during this validation pass (not an original review finding)

Symptom: `test_unhandled_exception_end_to_end_logs_detail_but_client_gets_generic_error` passed reliably in isolation but failed intermittently as part of the full suite, with `assert len(error_events) == 1` failing (`0 == 1` — the expected JSON log line was simply never captured).

Root cause, empirically confirmed (reproduced directly against structlog 24.4.0's `BoundLoggerLazyProxy.bind()` source, not guessed): with `cache_logger_on_first_use=True` (the app's real, intentional production setting — a legitimate performance optimization, not a defect), the *first* time any specific `structlog.get_logger(name)` proxy object is actually used to log something, that proxy permanently

monkeypatches its own instance-level `.bind` attribute to a closure bound to whatever the global structlog config was *at that exact moment*. Every later call to `structlog.configure()` is thereafter silently ignored by that specific, already-used proxy object — forever, for the lifetime of the process. `app.main.logger` (`structlog.get_logger("app")`, a module-level singleton) gets first-used the moment the *first* test anywhere in the whole suite starts up a `TestClient(app)` — almost always long before `test_exception_logging.py` even runs — locking it to whatever dev-mode/`ConsoleRenderer` configuration was active at that point. The test's own fixture correctly called `configure_logging()` to switch to the production JSON pipeline, but that call had zero effect on the already-cached `app.main.logger` proxy specifically — it only affected loggers not yet used.

This was independently verified with a minimal reproduction script (`structlog.configure()` → `get_logger()` → use once → `configure()` again with a distinguishing marker → use the *same* logger object again → the second use still emits the *first* configuration's marker; a brand-new `get_logger()` call at that point correctly picks up the new configuration, proving the caching is per-already-used-instance, not global).

Fix (test-only, no production code changed): `tests/security/test_exception_logging.py`'s `production_json_logging` fixture now explicitly resets `app.main.logger`'s cached `.bind` (via `del logger.bind`, which restores the class-level `bind` and forces it to re-derive from current global config on next use) immediately after each `configure_logging()` call, both on setup and on teardown. `app/core/logging.py`'s `cache_logger_on_first_use=True` setting itself was **not changed** — it remains correct and intentional for production, which never dynamically reconfigures logging mid-process and therefore never hits this failure mode in real operation. This is purely a test-harness fix for a test that dynamically reconfigures the process-global logging pipeline mid-suite, which no production code path ever does.

Validation of the fix: 5 consecutive full-suite runs, all 139 passing with no flakes (§14). The fix was also validated by deliberately running `test_exception_logging.py` both in isolation and combined with the rest of the suite in the same process — the previously-observed failure mode no longer reproduces in either configuration.

---

13. RLS Decision Note Correction

`docs/reports/Part-03.1/RLS-Decision-Note.md` previously stated, in two places, that "every query built by `BaseRepository`..." / "every existing tenant-owned query, without exception, goes through `BaseRepository`" as the description of the *current, live* state of tenant-isolation enforcement — exactly the inaccuracy the independent review's §22 identified. The note has been corrected (a visible correction banner plus rewritten §2, §4's cost/benefit table, §5, and §6):

- **Current reality**, now stated accurately: `BaseRepository` is a tested, tenant-aware repository foundation with **zero live application callers** today. The only live tenant-touching code (`auth_service.py`) uses direct, hand-written queries scoped by primary key or token hash — never an unscoped tenant list. The schema layer (composite FKs) and authorization layer (`get_current_active_user`) are the only tenant-isolation controls actually exercised by a live request right now.
- **The decision itself is unchanged** — RLS is still deferred — but for the corrected, more specific reason: there is currently no live, reachable, tenant-scoped **list/query** endpoint of *any* kind (repository-backed or not) for RLS to add value against, not because `BaseRepository` already universally mediates every query.
- **Future requirement**, made explicit: the first PR that ships a real tenant-scoped list/CRUD route to production is now the single most concrete, nearest-term trigger condition to revisit RLS (added to §6, per the independent review's own recommendation), regardless of whether that route is built on `BaseRepository` or a hand-written query. The existing trigger conditions (admin BI tool, background workers, security incident, code-review-scale growth) are retained but reworded to no longer assume `BaseRepository` is the only possible centralized mechanism.

No change was made to the underlying RLS decision (defer) — only to the accuracy of its stated justification, per instruction.

---

14. Files Changed

New files (this remediation pass):

- `backend/app/core/client_ip.py` — centralized trusted-proxy resolution (SEC-001/SEC-003).

- `backend/tests/security/test_client_ip.py` — 18 unit tests.
- `backend/tests/security/test_trusted_proxy_http.py` — 10 end-to-end HTTP tests (also covers TEST-002/TEST-003).
- `backend/tests/security/test_exception_logging.py` — 5 tests (OBS-001).
- `backend/tests/security/test_audit_logging.py` — 8 tests (TEST-001).

Modified files (this remediation pass):

- `backend/app/core/config.py` — added `TRUSTED_PROXY_MODE`, `TRUSTED_PROXY_NETWORKS`, validator, production-config check.
- `backend/app/core/rate_limit.py` — keys on `resolve_client_identity(request).resolved_ip` instead of `request.client.host`.
- `backend/app/middleware/security_headers.py` — HSTS scheme now sourced from `resolve_client_identity`; header application extracted to `apply_baseline_security_headers()`.
- `backend/app/api/v1/auth.py` — `_client_meta()` sources IP from `resolve_client_identity`.
- `backend/app/core/logging.py` — added `structlog.processors.format_exc_info` to the production renderer chain.
- `backend/app/middleware/request_context.py` — stores `request.state.request_id` in addition to the contextvar.
- `backend/app/main.py` — `unhandled_handler` reads `request.state.request_id`, passes it explicitly to `logger.exception`, and calls `apply_baseline_security_headers` on the 500 response.
- `backend/app/models/user.py` — `User.roles` relationship gains `order_by="Role.name"` (SEC-002).
- `backend/app/services/auth_service.py` — module docstring corrected (ARCH-001); `_primary_role_name` comment added (SEC-002).
- `backend/app/dependencies/auth.py` — comment added at the `roles[0]` call site (SEC-002).
- `backend/tests/security/test_authorization.py` — 1 new regression test (SEC-002 determinism).
- `docs/reports/Part-03.1/RLS-Decision-Note.md` — corrected per §13.

Total new/changed test count added in this pass: **42** (18 + 10 + 5 + 8 + 1).

---

15. Tests Added

File	New tests	Findings covered
`tests/security/test_client_ip.py`	18	SEC-001, SEC-003
`tests/security/test_trusted_proxy_http.py`	10	SEC-001, SEC-003, TEST-002, TEST-003
`tests/security/test_exception_logging.py`	5	OBS-001
`tests/security/test_audit_logging.py`	8	TEST-001
`tests/security/test_authorization.py` (+1 to existing 11)	1	SEC-002
Total new	**42**	

---

16. Validation Commands and Results

All commands below were actually executed in this session, in the `backend/` directory with the project's `.venv` activated. Exact output is summarized; nothing here is claimed without having been run.

```
$ ruff check .
All checks passed!

$ mypy app
Success: no issues found in 52 source files
```

```
$ python -m pytest -q -p no:warnings         # run 1
139 passed in 4.52s
$ python -m pytest -q -p no:warnings         # run 2
139 passed in 4.60s
$ python -m pytest -q -p no:warnings         # run 3
139 passed in 4.42s
$ python -m pytest -q -p no:warnings         # run 4
139 passed in 4.56s
$ python -m pytest -q -p no:warnings         # run 5
139 passed in 4.39s

$ python -m pytest -q -p no:warnings tests/security/
85 passed in 2.20s

$ python -m pytest -q -p no:warnings tests/security/test_client_ip.py tests/security/test_trusted_proxy_http.py
28 passed in 0.42s

$ python -m pytest -q -p no:warnings tests/security/test_exception_logging.py -v
5 passed in 0.31s

$ python -m pytest -q -p no:warnings tests/security/test_audit_logging.py -v
8 passed in 1.15s

$ python -m pytest -q -p no:warnings tests/security/test_authorization.py
12 passed in 0.86s

$ python -m pytest -q -p no:warnings tests/test_auth_service.py
12 passed in 1.36s
$ python -m pytest -q -p no:warnings tests/security/test_rate_limiting.py
4 passed in 0.13s
$ python -m pytest -q -p no:warnings tests/test_tenant_isolation.py
16 passed in 0.80s
$ python -m pytest -q -p no:warnings tests/security/test_cross_tenant_relationships.py
4 passed in 0.39s
$ python -m pytest -q -p no:warnings tests/security/test_session_security.py
2 passed in 0.44s
$ python -m pytest -q -p no:warnings tests/security/test_mass_assignment.py
4 passed in 0.32s
$ python -m pytest -q -p no:warnings tests/security/test_request_context.py tests/test_health.py
10 passed in 0.33s

$ alembic check                              # mislabeled — see correction banner at the top of
                                              # this report / Part-03.1-Minor-Security-Migration-Follow-Up.md.
                                              # This command actually ran against the `nexus` test
                                              # database (set earlier in the same shell session by a
                                              # prior test invocation's os.environ.setdefault), not the
                                              # live dev database (.env DATABASE_URL -> `abos`) as labeled.
                                              # `abos` was independently found to be two migrations
                                              # behind head at this point in time and has since been
                                              # brought to head separately — see the follow-up report.
No new upgrade operations detected.

# Fresh scratch database ("remediation_scratch"), created via a superuser role available
# in this sandbox (the app's own `nexus` role lacks CREATEDB):
$ alembic upgrade head
Running upgrade  -> 0a961c2ea2cf, initial schema
Running upgrade 0a961c2ea2cf -> 382d8497298e, add sessions and auth_tokens tables
Running upgrade 382d8497298e -> 3a2aec52f6d2, fix pre-existing created_by updated_by fk drift
Running upgrade 3a2aec52f6d2 -> 217e90d00cc0, cross tenant composite fks and soft delete safe unique indexes
$ alembic current
217e90d00cc0 (head)
$ alembic downgrade base
Running downgrade 217e90d00cc0 -> 3a2aec52f6d2 -> 382d8497298e -> 0a961c2ea2cf -> (empty)
$ alembic upgrade head               # round-trip
Running upgrade  -> 0a961c2ea2cf -> 382d8497298e -> 3a2aec52f6d2 -> 217e90d00cc0
$ alembic check
No new upgrade operations detected.
# scratch database dropped after validation.
```

Migration-state confirmation (regression review item — "migration state is unchanged unless a migration was genuinely required"): no migration was created or applied by this pass. The `nexus` database used by the test suite is confirmed at `217e90d00cc0 (head)`, unchanged. A separate local dev database (`abos`, referenced

only by `.env`, unrelated to the test suite) was found at an earlier revision (`382d8497298e`) — this is **pre-existing state from before this session**, not something altered here; it was not touched by any command in this pass (`alembic upgrade` was never run against it).

---

17. Regression Review

Item	Result
Existing authentication behavior	Correct — `tests/test_auth_service.py` (12/12), `tests/security/test_authorization.py` (12/12, including the new SEC-002 test) pass.
Existing rate limiting	Correct — `tests/security/test_rate_limiting.py` (4/4) pass; Redis fail-open path (`except RedisError: ... return`) confirmed byte-for-byte unchanged by reading the current file.
Redis fail-open behavior remains intentional	Confirmed — unchanged code, unchanged tests, unchanged documented rationale in the module docstring.
Audit logging still works	Confirmed and now positively tested — 8/8 new tests in `test_audit_logging.py`, plus the pre-existing functional wiring untouched.
Request-ID sanitization still works	Confirmed — `tests/security/test_request_context.py` (part of 10/10 passing with `test_health.py`).
Security headers still work	Confirmed — `tests/security/test_trusted_proxy_http.py::test_baseline_security_headers_present_on_a_real_response` and the OBS-001 500-path test both pass.
Tenant-isolation tests still pass	Confirmed — `tests/test_tenant_isolation.py` (16/16), `tests/security/test_cross_tenant_relationships.py` (4/4).
Migration state unchanged unless genuinely required	Confirmed — see §16; zero migrations created or applied.
No secrets introduced	Confirmed — diff scanned for password/secret/token/api-key literal patterns; only match was a pre-existing test-fixture default password string (`"Secret123x"`), not a real credential.
No test-only security bypass can silently affect production	Confirmed — `RATE_LIMIT_ENABLED` defaults to `True` in `Settings`; it is only forced to `false` via `tests/conftest.py`'s env-var default and per-test `monkeypatch` (auto-reverted), never in application defaults or production config.

---

18. Remaining Findings

Per §3: ARCH-002, DB-001, and TEST-004 (all LOW severity) were **not** addressed in this pass — they were outside the required/optional scope defined for this remediation (the instruction limited optional work to ARCH-001, SEC-002, TEST-002, TEST-003 specifically). They remain open findings from the independent review, unchanged

by this pass.

19. Deferred Items

- Cloudflare/Akamai edge integration — explicitly deferred per instruction; `app/core/client_ip.py` is designed to accept this later as configuration (`TRUSTED_PROXY_NETWORKS` population, possibly a new `TRUSTED_PROXY_MODE` value for header-based schemes), not a rewrite.
- PostgreSQL Row-Level Security — remains deferred, per the corrected `RLS-Decision-Note.md` (§13); the concrete trigger ("first tenant-scoped list/CRUD route ships") is now explicit.
- A true privilege-ordered "primary role" concept for multi-role users — SEC-002's fix is determinism-only, by design; an actual privilege hierarchy (if ever needed) is a new, larger design decision explicitly out of this pass's scope.
- ARCH-002, DB-001, TEST-004 — see §18.

20. Residual Risks

- **SEC-002's determinism is alphabetical, not privilege-based.** If multi-role assignment ships in the future with, e.g., both "Admin" and "Owner" roles on one user, `roles[0]` will deterministically resolve to `"admin"` (alphabetically first) — which happens to be reasonable for the current seeded role set (`Owner`/`Admin`/`Manager`/`Employee`/`Viewer`, both privileged roles hold `"*"`), but this is a coincidence of the current naming, not a designed guarantee. Whoever builds multi-role assignment must revisit whether alphabetical-by-name is still the right choice, or whether an explicit priority is needed by then.
- **TRUSTED_PROXY_NETWORKS is currently empty / `TRUSTED_PROXY_MODE=none` in every environment this pass tested against** (safe default). The actual Railway (or other) production edge/proxy CIDR ranges have not been identified or configured in this repository — deploying to a real reverse-proxied environment without setting these will silently fall back to "no proxy trust," which is *safe* (no header spoofing possible) but means rate limiting will still collapse to one bucket per proxy IP in that specific topology until an operator explicitly configures the real ranges. This is a deployment-configuration task, not a code defect, and was called out in the code's own comments (`config.py`) to make the required next step unambiguous.
- **The `cache_logger_on_first_use` test-isolation issue (§12.1) is specific to this test file's design** (dynamically reconfiguring the global structlog pipeline mid-suite). Any *future* test that also needs to reconfigure logging mid-session and assert against a logger that other, earlier-running tests may have already used will need the same `del logger.bind` pattern, or an equivalent. This is now documented in the fixture's own docstring for the next engineer who encounters it.
- **No production deployment exists to validate any of this against.** All verification in this pass (as in the original hardening pass) was against local PostgreSQL/Redis and `TestClient`. `validate_production_config()`'s new `TRUSTED_PROXY_MODE=static`-with-empty-list check was verified via unit tests constructing `Settings` objects, not an actual production boot.

21. Recommended Re-Review Scope

For the next independent re-review, the following should be specifically re-verified (in addition to a general pass):

- **SEC-001/SEC-003:** independently re-derive the right-to-left `X-Forwarded-For` algorithm from `client_ip.py` and confirm it matches the documented invariant; independently attempt the "attacker sends `X-Forwarded-For: 1.1.1.1` directly" spoof against a configured trusted-proxy test setup and confirm it fails as described in §5.
- **OBS-001:** independently reproduce the end-to-end test's log capture and confirm the JSON line genuinely contains a real traceback, not just that the test's own assertions pass (i.e., read the captured JSON output directly).
- **TEST-001:** independently query the test database during/after a run of `test_audit_logging.py` to confirm the asserted rows are real, not artifacts of the test's own mocking.
- **SEC-002:** confirm the `order_by="Role.name"` fix is still in place if `models/user.py` is touched by any future PR, since nothing currently prevents a future edit from silently dropping it (no dedicated migration or schema-level enforcement — it is a Python-level ORM concern only).

• **The RLS-Decision-Note.md correction (§13):** confirm the corrected note is internally consistent and that the "first tenant-scoped list/CRUD route" trigger is actually acted upon when that route is eventually built.

• **General:** re-run the exact validation commands in §16 independently rather than trusting this report's transcription of results.

22. Final Remediation Status

Finding	Status
SEC-001	**CLOSED**
OBS-001	**CLOSED**
TEST-001	**CLOSED**
SEC-003	**CLOSED**
ARCH-001	**PARTIALLY CLOSED** — documentation corrected; live-callers fact itself intentionally unchanged (by instruction)
SEC-002	**CLOSED**
TEST-002	**CLOSED**
TEST-003	**CLOSED**

This report does not change the independent engineering review's verdict. The verdict remains as issued (`APPROVED WITH REQUIRED REMEDIATION`) until an independent re-review evaluates the work described here. This document is a remediation record for that re-review to evaluate, not a self-certification of approval.